

第四章：文件系统

一、核心概念

1.1 文件抽象

"一切皆文件"是 Unix 哲学的核心。文件系统将持久化存储、设备、管道、网络连接等统一抽象为文件接口 (open/read/write/close) , 让用户程序无需关心底层差异。

文件的层次抽象：

```
用户视角:  open("/foo/bar.txt") → fd → read(fd, buf, n)
VFS 层:    inode_operations / file_operations
具体文件系统: ext4 / FAT32 / tmpfs / procfs
块缓存:    buffer cache (bread/bwrite)
块设备驱动: VirtIO / NVMe / SATA
```

1.2 inode

inode 是文件系统中文件的元数据描述符（不包含文件名）。一个 inode 通常包含：

- 文件类型（普通文件、目录、符号链接、设备文件）
- 大小
- 权限和所有者
- 时间戳（创建、修改、访问）
- 数据块的位置信息

文件名和 inode 的关系：目录是一种特殊文件，内容是（**文件名**，**inode号**）的映射表。一个 inode 可以被多个文件名引用（硬链接）。

1.3 VFS（Virtual File System）

VFS 是操作系统中文件系统的抽象层，定义统一的接口，让不同的具体文件系统（ext4、FAT32、NFS 等）通过相同的 API 供用户使用。

Linux VFS 四大核心对象：

对象	含义	关键操作
superblock	一个已挂载的文件系统实例	alloc_inode, write_inode
inode	一个文件的元数据	lookup, create, link, unlink
dentry	目录项缓存（路径→inode映射）	d_compare, d_hash
file	一个打开的文件实例	read, write, llseek, mmap

1.4 块缓存（Buffer Cache）

磁盘 I/O 很慢（机械硬盘 ~10ms，SSD ~0.1ms），远低于内存访问速度（~100ns）。块缓存在内存中保留最近访问的磁盘块，减少实际 I/O 次数。

写策略：

- **Write-through**：写入缓存同时写入磁盘（简单但慢）
- **Write-back**：只写入缓存，延迟写回磁盘（快但需要崩溃恢复机制）

1.5 日志/事务（Journaling）

文件系统操作通常涉及多个磁盘块的修改（例如创建文件需要修改 inode 块、目录块、位图块）。如果中间断电，文件系统会处于不一致状态。

日志的解决方案：

1. 将所有修改先写入日志区域（write-ahead log）
2. 日志写完后标记事务完成（commit）
3. 将修改应用到实际位置
4. 清除日志

如果断电发生在 commit 前：丢弃日志，文件系统未改变。如果断电发生在 commit 后：重放日志，恢复一致性。

1.6 FAT32

FAT（File Allocation Table）是最广泛兼容的文件系统，结构简单：

磁盘布局：

【保留区(BPB)】 【FAT 表1】 【FAT 表2】 【数据区(从簇2开始)】

FAT 表：簇号 → 下一个簇号 的数组

fat[2] = 3 // 簇 2 的下一个是簇 3

fat[3] = 0x0FFFFFFF // 簇 3 是文件末尾

簇 → 扇区：sec = first_data_sec + (clus - 2) * sec_per_clus

长文件名（LFN）：FAT32 原始只支持 8.3 短文件名。VFAT 扩展通过复用目录项格式，在正式目录项前放置多个 LFN 条目（每个存 13 个 Unicode 字符），倒序排列。

1.7 ext4

ext4 是 Linux 的主流文件系统，特性包括：

- **Extent 树**：取代 ext2/ext3 的间接块，一个 extent 描述一段连续的磁盘块
- **延迟分配**：写入时不立即分配磁盘块，合并后一次性分配大段连续空间
- **日志模式**：ordered（默认，元数据日志）/ journal（全日志）/ writeback
- **块组**：将磁盘分成多个块组，每个组有独立的 inode 表和数据块

二、基本推论

推论 1: VFS 抽象层必须足够灵活以容纳不同文件系统。 FAT32 没有 inode 概念, NFS 是网络文件系统, procfs 根本不在磁盘上。VFS 通过函数指针 (操作表) 实现多态, 每个文件系统注册自己的实现。

推论 2: 块缓存是文件系统性能的关键。 没有缓存, 每次 read() 都要等磁盘 I/O。有了缓存, 热数据的访问延迟从毫秒降到纳秒。Linux 的 page cache 把整个空闲内存都用作缓存。

推论 3: 日志的开销是写放大。 每个修改要写两次 (一次日志, 一次实际位置)。这是一致性的代价。现代文件系统通过只记录元数据日志 (ordered 模式) 来平衡性能和安全。

推论 4: FAT32 的简单性是双刃剑。 没有权限、没有日志、没有硬链接、簇链遍历是 $O(n)$ 。但它几乎被所有操作系统和设备支持, 是 U 盘和 SD 卡的默认格式。

推论 5: 文件描述符是 "一切皆文件" 的关键。 fd 是用户态看到的整数句柄, 内核通过 fd 查找 `struct file`, `struct file` 指向 inode 或其他后端 (pipe、socket、eventfd)。这种间接层让 dup、fork 的语义自然正确。

三、Linux / 通用实现

3.1 Linux VFS 架构

```

用户空间:  open() / read() / write() / close()
              ↓ syscall
VFS 层:    do_sys_open → path_lookup → dentry 缓存
              vfs_read → file->f_op->read
              ↓ 分发到具体文件系统
具体 FS:   ext4_file_read_iter / fat_file_read
              ↓
块层:      submit_bio → I/O 调度器 → 块设备驱动

```

函数指针虚表机制:

```

struct inode_operations {
    struct dentry *(*lookup)(struct inode *, struct dentry *, unsigned
int);
    int (*create)(struct inode *, struct dentry *, umode_t, bool);
    int (*unlink)(struct inode *, struct dentry *);
    // ...
};

struct file_operations {
    ssize_t (*read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write)(struct file *, const char __user *, size_t, loff_t
*);
    int (*mmap)(struct file *, struct vm_area_struct *);
    // ...
};

```

每个文件系统在注册时填充这些函数指针。VFS 调用 `f_op->read()` 就会自动分发到对应文件系统的实现。

3.2 Linux Page Cache

Linux 不使用独立的 buffer cache，而是通过 **page cache** 统一管理文件数据缓存：

- 文件的每个页在 page cache 中有一份缓存
- `read()` → 检查 page cache → 命中则直接拷贝 → 未命中则读磁盘
- `write()` → 写入 page cache → 标记 dirty → 后台 writeback 线程定期刷盘

3.3 Linux 的文件描述符层

进程： task_struct → files_struct → fdtable

↓

fd: [0] → struct file (stdin)
 [1] → struct file (stdout)
 [2] → struct file (stderr)
 [3] → struct file (/foo/bar.txt)

↓

f_inode → inode (文件元数据)
 f_op → file_operations (读写等操作)
 f_pos → 当前偏移

`dup()` 创建新 fd 指向同一个 `struct file` (共享 `f_pos`)。 `fork()` 复制 fd 表，每个 `struct file` 的引用计数 +1。

3.4 getdents64

`ls` 命令通过 `getdents64()` 读取目录内容：

```
struct linux_dirent64 {
    uint64 d_ino;        // inode 号
    int64  d_off;        // 下一个 dirent 的偏移
    uint16 d_reclen;      // 本 dirent 的总长度
    uint8  d_type;        // 文件类型 (DT_REG/DT_DIR/...)
    char   d_name[];      // 文件名 (变长)
};
```

四、RUOS 的实现

核心文件

- `src/fs/fs.c` — VFS 分发层
- `src/fs/fat32.c` — FAT32 完整实现
- `src/fs/ext4fs.c` — ext4 (基于 `lwext4` 库)
- `src/fs/file.c` — 文件描述符管理
- `src/fs/pipe.c` — 管道

- `src/fs/eventfd.c` — eventfd
- `src/fs/procfs.c` — /proc 虚拟文件系统
- `src/fs/bio.c` — 块缓存层
- `src/fs/log.c` — 日志事务层 (xv6fs)

4.1 三层 VFS 自动探测

比赛测评环境可能给 xv6fs、FAT32 或 ext4 镜像，内核必须自动适应。

```
fsinit(dev):
    1. ext4fs_init(dev) → 成功 → ext4_mode = 1, return
    2. fat32_init(dev)  → 成功 → fat32_mode = 1, return
    3. 读 superblock    → magic 匹配 → xv6fs, return
```

运行时分发用全局 flag + if-else:

```
readi(ip, ...):
    if (ip->major == FAT32_INODE_TAG) → fat32_readi(...)
    else if (ext4_mode)                → ext4_readi(...)
    else                               → xv6_readi(...)
```

对比 Linux 的 VFS: Linux 用函数指针虚表 (`inode_operations/file_operations`) 实现多态。RUOS 用 if-else 分发——代码量最小，调试最容易，但扩展性差（每加一个文件系统要改所有分发点）。

设计理由: 教学 OS 只需要支持 3 种文件系统，函数指针虚表的工程开销不值得。

4.2 FAT32 实现

伪 inode 统一: FAT32 没有 inode 概念，RUOS 用 `make_inode()` 构造伪 inode:

- `inum` = 簇号
- `major` = `FAT32_INODE_TAG` (标记路径)
- `addrs[0]` = 起始簇号

上层代码拿到 `struct inode*` 后，`readi/writei` 检查 `major` 走 FAT32 路径。

LFN 解析: 每个 LFN 项存 13 个 Unicode 字符（分散在 3 个固定偏移位置），段序号 1-based 且倒序存储。非 ASCII 字符替换为 ?。

512 字节扇区 vs 1024 字节 BSIZE:

```
read_sector512(dev, sec512):
    block1024 = sec512 / 2;
    buf = bread(dev, block1024);
    offset = (sec512 % 2) * 512;
    return buf->data + offset;
```

4.3 ext4 支持

自己实现 ext4 不现实（extent 树、日志、块组等极其复杂），RUOS 嵌入了 **lwext4** 库——一个面向嵌入式的 ext4 实现。

适配工作：为 lwext4 实现块设备接口（bread/bwrite 桥接），配置只读/读写模式，处理 inode 到 VFS 层的转换。

4.4 eventfd

比赛决赛题目之一。用于进程/线程间的轻量事件通知。

```
struct eventfd {
    struct spinlock lock;
    uint64 counter;      // 核心：64 位计数器
    int flags;           // EFD_NONBLOCK | EFD_SEMAPHORE | EFD_CLOEXEC
    int refcount;
};
```

- **write(fd, &val, 8)**: counter += val
- **read(fd, &buf, 8)**: 普通模式取走全部 (buf=counter, counter=0)；信号量模式每次取 1
- 集成为 **struct file** (type=FD_EVENTFD)，复用 fd 体系

对比 pipe：eventfd 没有缓冲区拷贝，只有一个原子计数器。适合纯事件通知场景。

4.5 procfs

实现了虚拟 **/proc** 文件系统，不在磁盘上，数据由内核动态生成。用于暴露内核状态给用户态。

对比 Linux：Linux 的 procfs 极其丰富（**/proc/pid/maps**、**/proc/meminfo**、**/proc/cpuinfo** 等）。RUOS 的 procfs 是最小实现，主要用于测试兼容。

五、八股 / 面试高频问题

Q: 硬链接和软链接（符号链接）的区别？ A: 硬链接是同一 inode 的多个目录项（文件名），共享数据。软链接是独立的 inode，内容是目标路径字符串。硬链接不能跨文件系统，不能链接目录；软链接都可以。删除原文件，硬链接仍可访问，软链接变成悬空。

Q: inode 是什么？ A: 文件的元数据描述符——存储类型、大小、权限、数据块位置等。文件名不在 inode 中，而在目录项中。一个 inode 可被多个文件名引用（硬链接），inode 的 link count 记录引用数。

Q: ext4 的 extent 和 ext2 的间接块有什么区别？ A: ext2 用直接块+间接块+二级间接+三级间接寻址数据块，大文件需要多级间接跳转。ext4 用 extent（起始块+长度），一个 extent 描述一段连续磁盘块，大幅减少元数据量，提高顺序 I/O 性能。

Q: 什么是日志文件系统？为什么需要日志？ A: 文件操作涉及多个磁盘块修改。中间断电会导致不一致（如 inode 已更新但数据块未写入）。日志先将修改写入日志区域，commit 后再写入实际位置。断电恢复时重放或丢弃日志。

Q: Linux VFS 的四大核心对象？ A: superblock（挂载的文件系统实例）、inode（文件元数据）、dentry（目录项缓存，路径→inode 映射）、file（打开的文件实例）。每个对象有对应的操作表（函数指针），实现多态。

Q: 文件描述符、打开文件表、inode 的关系？ A: fd 是进程级的整数索引 → 指向系统级的 struct file（包含 offset、权限） → 指向 inode（文件元数据）。dup 共享 struct file（共享 offset），fork 复制 fd 表但 struct file 引用计数+1。

Q: 什么是 Page Cache？ A: Linux 将文件数据缓存在内存中（以页为单位）。read 先查 page cache，命中则直接拷贝；miss 则读磁盘并缓存。write 写入 page cache 标记 dirty，后台 writeback 线程定期刷盘。空闲内存自动用作 cache。

六、项目贡献亮点

- 实现了三层 VFS 自动探测（ext4 > FAT32 > xv6fs），适配比赛不同格式的磁盘镜像
- 完整实现 FAT32（BPB 解析、簇链遍历、LFN/SFN、目录扩展），理解 FAT 表和簇链的工作原理
- 通过嵌入 lwext4 库支持 ext4，理解 OS 与第三方库的适配工作
- 实现 eventfd2（决赛题目），集成到文件描述符体系，支持普通/信号量/非阻塞三种模式

七、设计取舍总结

决策	RUOS 选择	Linux 做法	权衡
VFS 架构	全局 flag + if-else	函数指针虚表	简单但不可扩展
FAT32 inode	伪 inode + major tag	vfat_iget	最小改动复用 xv6 缓存
ext4 支持	嵌入 lwext4 库	内核原生实现	自己写不现实
块缓存	固定大小 buffer cache	page cache（统一）	简单但不共享内存
块大小	BSIZE=1024 + 512 桥接	4KB page cache	兼容 xv6 原有设计
日志	xv6 原版 log（仅 xv6fs）	ext4 ordered 日志	FAT32 无日志保护
文件描述符上限	128 (NOFILE)	动态 fdtable	固定但 BusyBox 够用